Programación para la Ciencia de Datos: Informe Técnico de Metodologías de Manipulación de Datos

Integrantes:

- David Ramírez
- Octavio Chavez
- Felipe Huincaman

Asignatura: Programación para la Ciencia de Datos, sección 1D
Docente: Profesora Jazna Meza
Fecha: 8 de abril, 2026

Índice

1. Resumen Ejecutivo 2
2. Análisis Exploratorio Inicial 3
    Análisis de tipos de datos de cada columna 3
    Análisis de estadísticas descriptivas iniciales de variables numéricas 3
    Análisis de valores en columnas cualitativas 4
    Análisis de valores nulos 4
    Análisis de valores duplicados 4
    Análisis de valores atípicos 5
    Análisis de distribución inicial de columnas numéricas 6
    Conclusiones del análisis inicial 6
3. Metodología de Transformación 7
    Pipelines de limpieza y transformación 7
    Conversión de tipos 7
    Imputación de valores faltantes 7
    Tratamiento de outliers 8
    Normalización de características numéricas 8
    Tratamiento de duplicados 8
    Codificación para variables nominales y ordinales 8
    Ingeniería de Características (Feature Engineering) 9
    Conclusiones de la etapa de limpieza y transformación 9
4. Resultados y Validaciones 10
    Integridad y calidad del nuevo conjunto de datos 10
    Control de distribuciones y atípicos 11
    Validación de Codificación 11
5. Conclusiones y recomendaciones 12
    Reflexión final 12
    Dificultades encontradas 12

1. Resumen Ejecutivo

El presente informe técnico documenta la fase inicial y más importante en la construcción de una solución analítica para un negocio: la preparación de los datos. Nuestro equipo de trabajo de científicos de datos en formación, ha sido presentado a la problemática de una empresa de servicios digitales que enfrenta el aumento en la tasa de abandono de clientes, lo cual impacta directamente en la rentabilidad del negocio. En este contexto, nuestro equipo ha procesado un extenso conjunto de información de los usuarios, trabajando bajo estándares y aplicando buenas prácticas de un entorno profesional. Se han aplicado técnicas avanzadas de manipulación, limpieza y transformación de datos para asegurar la calidad e integridad de la información, con la intención de lograr una estrategia basada en datos para anticipar el comportamiento de los usuarios y evitar cancelaciones.

Durante esta oportunidad, el desarrollo de este proyecto contempla el uso de tecnologías clave en el ámbito de la ciencia de datos tales como Python, librerías de código como numpy, pandas y sklearn, y herramientas de trabajo colaborativo como Jupyter Notebook, Google Colab, Git y Github.

El resultado de este proyecto es generar un conjunto de datos robusto y listo para el desarrollo de modelos predictivos que ayuden al negocio a diseñar campañas de retención, optimizar sus costos de operación y ayudar en la toma de decisiones estratégicas.

2. Análisis Exploratorio Inicial

En esta sección se presenta el Análisis Exploratorio de Datos. A través de estadísticas descriptivas y apoyo visual, se caracterizan las variables para identificar tendencias, valores atípicos y la estructura general de la información.

Análisis de tipos de datos de cada columna

Invocar el método .info() en el dataset inicial entrega información detallada de los nombres de las columnas, tipos de datos de cada variable, el tamaño de la muestra y columnas donde hay datos faltantes, confirmando la necesidad de realizar tratamiento de datos.

```
<class 'pandas.DataFrame'>
RangeIndex: 20400 entries, 0 to 20399
Data columns (total 22 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id_cliente             20400 non-null  int64
 1   fecha_registro         20400 non-null  str
 2   edad                   20400 non-null  int64
 3   genero                 20400 non-null  str
 4   region                 20400 non-null  str
 5   estado_civil           20400 non-null  str
 6   ingreso_mensual        19379 non-null  float64
 7   gasto_mensual          19375 non-null  float64
 8   deuda_total            20400 non-null  float64
 9   score_crediticio       19379 non-null  float64
 10  antiguedad_meses       20400 non-null  int64
```

```
 10  antiguedad_meses       20400 non-null  int64
 11  frecuencia_compra      20400 non-null  int64
 12  ultima_compra_dias     20400 non-null  int64
 13  uso_app                20400 non-null  str
 14  tipo_plan              20400 non-null  str
 15  num_productos          20400 non-null  int64
 16  tiene_tarjeta_credito  20400 non-null  int64
 17  canal_registro         20400 non-null  str
 18  dia_semana_registro    20400 non-null  str
 19  hora_registro          20400 non-null  int64
 20  codigo_postal          20400 non-null  int64
 21  abandono               20400 non-null  int64
dtypes: float64(4), int64(10), str(8)
memory usage: 3.4 MB
```

Figuras 1 y 2: data.info()

Durante este análisis inicial, se identificaron variables que dentro del conjunto de datos existen como numéricas booleanas representadas como 0 y 1, pero se interpretan como categóricas, es decir confirman o niegan una característica del usuario. Estas variables son ‘Tiene Tarjeta Crédito’ y ‘Abandono’. Además, se encontraron otras variables cuantitativas que si bien están representadas como números, no poseen poder analítico para el presente estudio, estas variables son ‘ID Cliente’, ‘Fecha Registro’ y ‘Código Postal’.

Análisis de estadísticas descriptivas iniciales de variables numéricas

Aplicar el método .describe() al dataset, genera las estadísticas descriptivas presentes antes del tratamiento de los datos. Esta información es relevante ya que nos ayuda a tomar decisiones acerca de las variables que influyen en el target, su comportamiento y el tipo de limpieza y transformación que necesitan.

	edad	ingreso_mensual	gasto_mensual	deuda_total	score_crediticio	antiguedad_meses	frecuencia_compra	ultima_compra_dias	num_productos	tiene_tarjeta_
count	20400.00000	1.937900e+04	19375.000000	2.040000e+04	19379.000000	20400.000000	20400.000000	20400.000000	20400.000000	20400
mean	48.54598	8.330390e+05	399453.495188	2.061953e+06	600.441593	60.205882	9.966765	182.325686	2.994853	0
std	17.84971	4.301978e+05	150831.135171	1.055155e+06	100.408685	34.210052	5.460586	105.056827	1.410716	0
min	18.00000	-8.302577e+04	-194957.467079	-1.684124e+06	220.516968	1.000000	1.000000	1.000000	1.000000	0
25%	33.00000	6.346396e+05	296753.420113	1.461942e+06	532.872163	31.000000	5.000000	92.000000	2.000000	0
50%	48.00000	8.044483e+05	398887.260180	2.017759e+06	600.955064	61.000000	10.000000	181.000000	3.000000	1
75%	64.00000	9.737937e+05	500485.426137	2.557871e+06	668.245607	90.000000	15.000000	274.000000	4.000000	1
max	79.00000	7.560936e+06	954575.217321	1.777694e+07	998.290682	119.000000	19.000000	364.000000	5.000000	1

Figura 3: data.describe(). Se omiten las columnas de ID y Código Postal.

Análisis de valores en columnas cualitativas

Invocar el método .describe() e incluir la palabra clave ‘object’ genera información detallada acerca de las variables categóricas que existen dentro de nuestro conjunto de datos. Esta información es el tamaño de la columna, cuántos valores únicos existen y su frecuencia. Además invocar el método unique(), devuelve los valores únicos en estas variables categóricas.

	fecha_registro	genero	region	estado_civil	uso_app	tipo_plan	canal_registro	dia_semana_registro
count	20400	20400	20400	20400	20400	20400	20400	20400
unique	2000	3	3	3	3	3	3	7
top	2018-06-14	Otro	Sur	Casado	Bajo	Estandar	App	Miercoles
freq	21	6997	6812	6828	6886	6878	6894	3000

Figura 4: data.describe(include="object")

Se identificaron variables cualitativas ordinales, ‘Uso App’ y ‘Tipo Plan’ . Luego, existen variables cualitativas nominales, ‘Genero’, ‘Region’, ‘Estado Civil’, ‘Canal Registro’ y ‘Dia Semana Registro’. Basándonos en estos resultados, posteriormente se decidirá cómo usar las técnicas de codificación One-Hot u Ordinal según corresponda.

Análisis de valores nulos

Para determinar cuáles variables presentan valores faltantes, se hizo uso del método isnull().sum() sobre el dataset. Determinamos que ‘ingreso_mensual’ y ‘score_crediticio’ contienen la misma cantidad de nulos, y ‘gasto_mensual’ tiene una cantidad apenas superior.
Se requerirá aplicar una técnica de imputación para tratar los valores nulos.

```
1   id_cliente              0
2   fecha_registro          0
3   edad                    0
4   genero                  0
5   region                  0
6   estado_civil            0
7   ingreso_mensual      1021
8   gasto_mensual        1025
9   deuda_total             0
10  score_crediticio     1021
11  antiguedad_meses        0
12  frecuencia_compra       0
```

```
12  frecuencia_compra        0
13  ultima_compra_dias       0
14  uso_app                  0
15  tipo_plan                0
16  num_productos            0
17  tiene_tarjeta_credito    0
18  canal_registro           0
19  dia_semana_registro      0
20  hora_registro            0
21  codigo_postal            0
22  abandono                 0
```

Figuras 5 y 6: data.isnull().sum()

Análisis de valores duplicados

Aplicando el método .duplicated().sum() sobre el dataset entrega como resultado ‘np.int64(400)’, lo cual se interpreta como que existe un total de 400 valores duplicados que necesitan ser eliminados durante la etapa de limpieza en el tratamiento de los datos.

Análisis de valores atípicos

Para detectar la existencia de outliers, se han generado visualizaciones de gráficos boxplot, la cual es una técnica intuitiva útil que muestra los cuartiles y los valores atípicos mediante puntos fuera de los extremos. Una vez detectados los outliers, se van a realizar los cálculos pertinentes y decidir estrategias adecuadas para mantener la integridad de los datos.

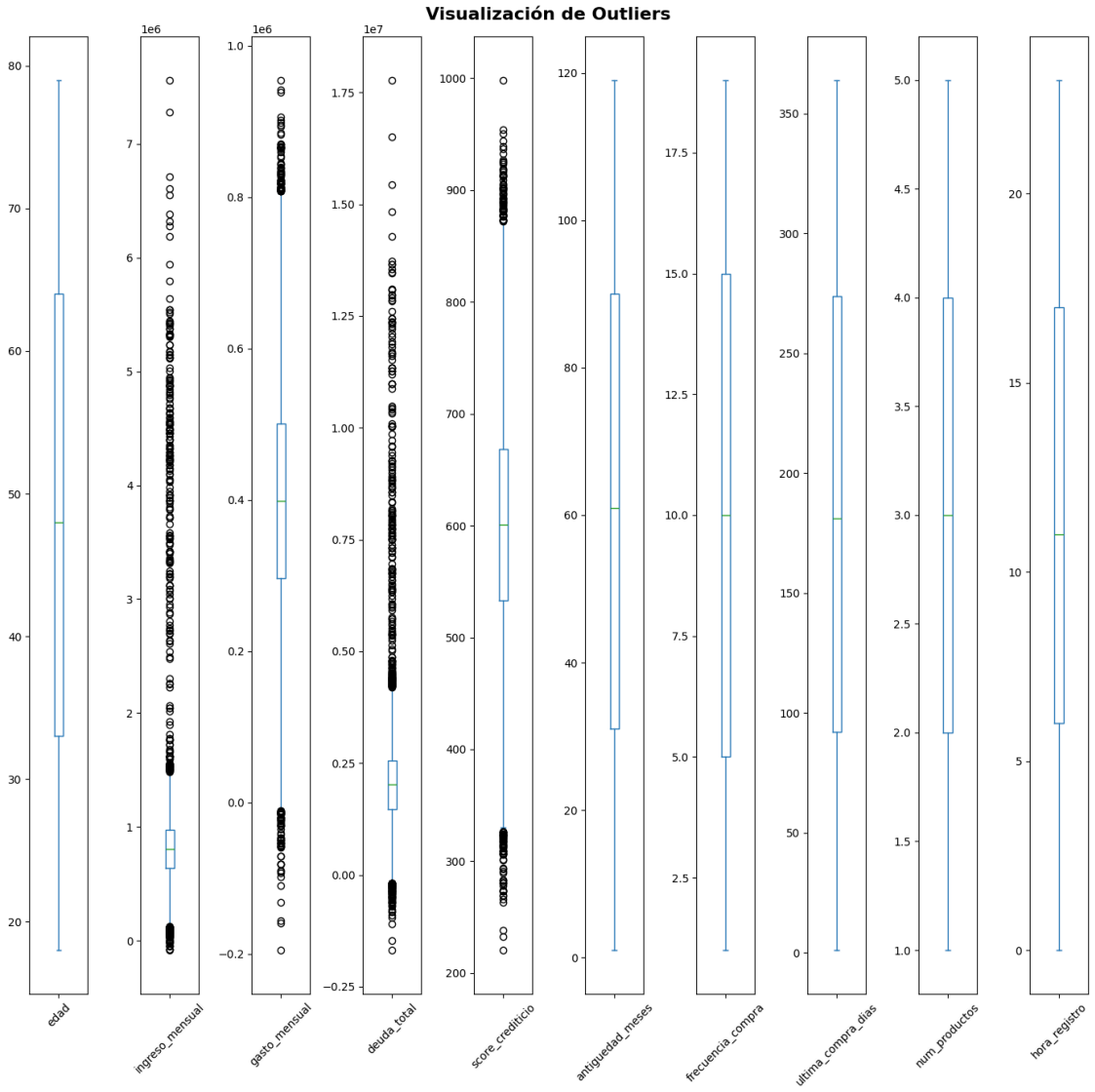
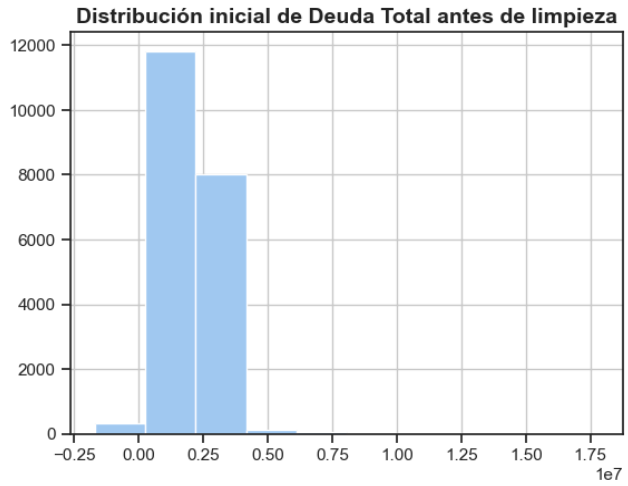


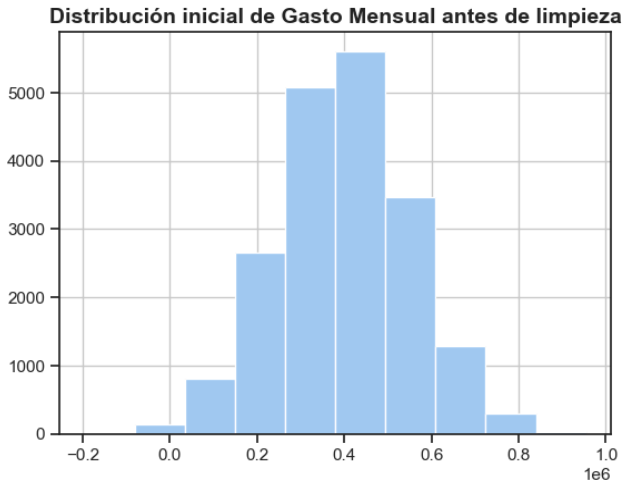
Figura 7: diagrama BoxPlot

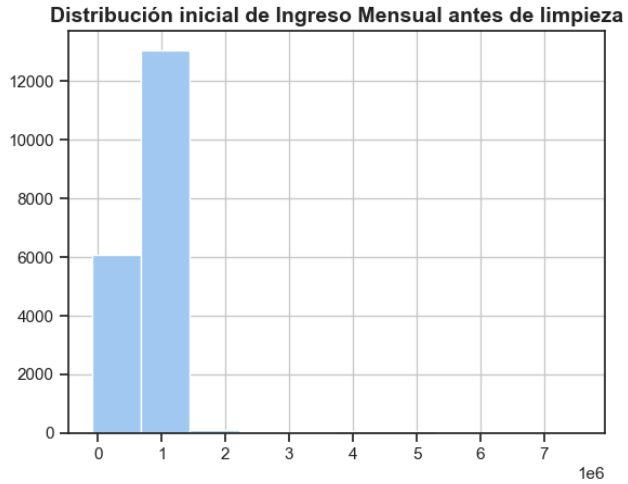
Se observa que ‘tiene_tarjeta_credito’ y ‘abandono’ no son candidatos para la búsqueda de atípicos, ya que cuentan como categóricos (booleanos). Además se puede identificar a las columnas ‘ingreso_mensual’, ‘gasto_mensual’, ‘deuda_total’ y ‘score_crediticio’ con alta probabilidad de tener datos atípicos.

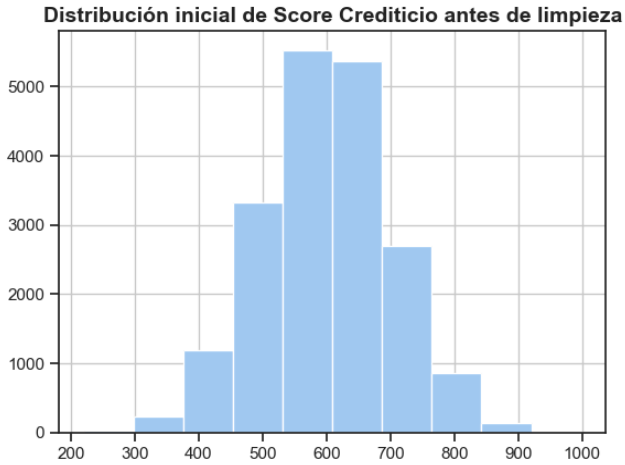
Análisis de distribución inicial de columnas numéricas

Se generan visualizaciones de cómo es la distribución inicial de principales variables numéricas que contienen datos sucios y serán sometidas a limpieza.









Conclusiones del análisis inicial

En síntesis, esta exploración inicial ha permitido diagnosticar el estado del conjunto de datos y definir los requerimientos técnicos para su adecuación. Como hallazgos principales, se detectaron 400 registros duplicados, valores nulos concentrados en variables económicas (ingreso mensual, score crediticio y gasto mensual) y una alta probabilidad de datos atípicos en las dichas variables financieras. Asimismo, se logró identificar columnas sin valor analítico como ID o Código Postal y se caracterizó la naturaleza de las variables útiles (booleanas, nominales y ordinales) para su correcto tratamiento.

3. Metodología de Transformación

La presente sección de Metodologías de Transformación detalla el tratamiento aplicado al conjunto de datos. A continuación, se explican las técnicas específicas empleadas para la limpieza, imputación y codificación de variables, acompañadas de la justificación técnica que respalda cada decisión.
Para comenzar, se clasifica y agrupa las variables del dataset en listas según su naturaleza estadística y el tratamiento específico que requieren. Esto permite construir un pipeline automatizado que aplicará la transformación o codificación exacta que cada grupo necesita.

Pipelines de limpieza y transformación

Durante la etapa de transformación de los datos utilizamos la metodología del uso de pipelines de transformación que consiste en la automatización de pasos de preprocesamiento y transformación de datos. Se utiliza principalmente para convertir datos crudos en un dataset preparado de forma eficiente, ordenada y sin errores. Un pipeline estándar en ciencia de datos enfocada a la transformación está compuesto por cuatro etapas fundamentales:

- Cargar: Entrada de los datos originales o crudos al sistema.
- Limpiar: Aplicación de filtros para corregir errores o datos faltantes.
- Transformar: Fase donde se aplican técnicas como la normalización, codificación de variables como One-Hot y Ordinal Encoder

¿Por qué es importante la transformación de datos? La transformación de datos ayuda a mejorar la calidad de los datos al mejorar su compatibilidad y usabilidad. Al estandarizar los líderes pueden estar seguros de que toman decisiones basadas en datos confiables, oportunos y precisos.

```
# Define pipeline para variables numéricas
pipeline_numerical_features = Pipeline(steps=[
    ('winsorizer', Winsorizer(limits=(0.05, 0.05))), # Aplica Winsorización para limitar outliers al 5%
    ('imputer', SimpleImputer(strategy='mean')), # Imputa valores faltantes con el promedio
    ('scaler', StandardScaler()) # Escala características numéricas
])
```

Pipeline para variables numéricas

Conversión de tipos

Convertir variables que inicialmente se representan como números a tipo categórico es importante para descartarlas del análisis de outliers, e impedir calcular medidas estadísticas que no tienen valor analítico para estas columnas. Este tratamiento se realiza para las variables booleanas ‘Tiene Tarjeta de Crédito’ y ‘Abandono’, aplicando el método .astype(‘category’).

Imputación de valores faltantes

Durante la etapa de limpieza, se define un paso de preprocesamiento identificado como ‘imputer’ que utiliza la clase SimpleImputer() para manejar los datos nulos en las variables numéricas. Al establecer strategy = ‘mean’, se llenan los datos faltantes con el promedio. Para

variables cuantitativas, el promedio es una medida muy sensible a los outliers, por lo tanto es más conveniente imputar por el promedio luego de haber hecho tratamiento de atípicos.
El estándar para imputar variables categóricas es utilizar la clase SimpleImputer para tratar los valores nulos, pero esta vez aplicando el parámetro strategy = 'most_frequent'. Esto logra que se llenen los datos faltantes utilizando la moda, o el valor que más se repite, dado que no es matemáticamente posible calcular un promedio o una mediana sobre texto.

Tratamiento de outliers

En este paso se aplica la técnica de Winsorización para manejar los valores atípicos. La Winsorización mitiga el efecto de los valores extremos sin necesariamente eliminar los registros. Al aplicar una Winsorización simétrica con la configuración limits=(0.05, 0.05) se identifican los valores que se encuentran en el 5% más bajo y en el 5% más alto de la columna, y los reemplaza por los valores exactos de los percentiles 5 y 95, respectivamente. Aplicar un recorte simétrico es la estrategia ideal para neutralizar estos valores extremos en ambos lados sin alterar la media.

Normalización de características numéricas

El último paso es la aplicación de la técnica de estandarización StandardScaler(). Su propósito es finalmente llevar los datos a una escala homogénea, con media de 0 y desviación estándar 1. Esto garantiza que al entrenar el modelo predictivo con nuestro conjunto de datos limpio, este evalúe el impacto de cada característica sin permitir que valores extremos afectan el análisis.

Tratamiento de duplicados

Se define una función personalizada 'eliminar_duplicados' que recibe un conjunto de datos y le aplica el método .drop_duplicates() de la librería Pandas. Su función es buscar todo el conjunto de datos y eliminar automáticamente aquellas filas que sean copias de otras, conservando solo la primera aparición.

```
# Define función para eliminar duplicados
def eliminar_duplicados(X):
  return X.drop_duplicates()
```

Figura 14: función eliminar_duplicados

Codificación para variables nominales y ordinales

Los modelos predictivos procesan números, pero las variables nominales no tienen un orden matemático, por ejemplo, el género 'Masculino' no vale más ni menos que el género 'Femenino'. El siguiente paso del pipeline es aplicar un OneHotEncoder() para variables

categóricas nominales, la codificación One-Hot crea una nueva columna binaria para cada categoría existente, garantizando que el modelo evalué sin asumir un orden falso.

```
# Define pipeline para variables categóricas nominales
pipeline_nominales = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Imputa valores faltantes con la moda
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # Aplica codificación OneHotEncoder para variables nominales
])
```

Figura 15: Pipeline para variables categóricas nominales

En variables ordinales, el orden si es importante por ejemplo Básico < Estándar < Premium.
A diferencia del caso anterior, este paso utiliza un OrdinalEncoder para las variables que sí poseen un orden claro como Tipo de Plan y Uso de App. A través del parámetro categories, utiliza listas previamente definidas 'orden_tipo_plan' y 'orden_uso_app' para forzar al modelo a respetar ese orden al momento de asignar números. Convertirlas a números enteros consecutivos (0, 1, 2) basándonos en nuestras listas predefinidas permite conservar este orden lógico. De este modo, el modelo de machine learning es capaz de interpretar la mejora entre una categoría y otra, aprovechando estas secuencias para mejorar sus predicciones.

```
# Define pipeline para variables categóricas ordinales
pipeline_ordinales = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Imputa valores faltantes con la moda
    ('ordinal', OrdinalEncoder(categories=[orden_tipo_plan, orden_uso_app])) # Aplica codificación OrdinalEncoder pa
])
```

Figura 15: Pipeline para variables categóricas ordinales

Ingeniería de Características (Feature Engineering)

Se determinó que variables numéricas financieras como 'Ingreso Mensual', 'Gasto Mensual' y 'Deuda Total' pueden influir en el comportamiento de abandono de los clientes. Por ende se procedió a definir nuevas columnas generadas a partir de cálculos usando las métricas antes mencionadas. Para lograr esto se creó una función 'agregar_variables', que a su vez genera las nuevas columnas 'ratio_endeudamiento' y 'porcentaje_gasto'.
La columna 'ratio_endeudamiento' es el cociente entre deuda_total e ingreso_mensual, y mide la carga financiera de un cliente, en comparación con su capacidad de generar recursos.
Luego, la columna 'porcentaje_gasto' es el cociente entre 'gasto_mensual' e 'ingreso_mensual' y mide el comportamiento de consumo de un cliente en relación con su capacidad económica.

Conclusiones de la etapa de limpieza y transformación

En síntesis, la preparación del conjunto de datos se ejecutó mediante códigos estructurados y reproducibles. Tras resolver la suciedad individual mediante técnicas de imputación, Winsorización, escalado y codificación One-Hot y Ordinal, el proceso terminó con la integración de pipelines de transformación en un único flujo de trabajo mediante la herramienta ColumnTransformer.

Además, se definió un pipeline de limpieza que incluye la eliminación de duplicados y el preprocesamiento específico para cada tipo de variable. Finalmente, se aplicó este pipeline de limpieza al dataset y se obtuvo como resultado una conjunto de datos íntegro y estandarizado, estableciendo una base sólida para la construcción de los modelos predictivos en la siguiente fase del proyecto.

4. Resultados y Validaciones

Posteriormente de haber aplicado la secuencia automatizada de pasos de preprocesamiento y transformación mediante pipelines a nuestro conjunto de datos obtenemos un dataset limpio, ordenado y listo para ser entregado al algoritmo de machine learning.

Integridad y calidad del nuevo conjunto de datos

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 33 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   ingreso_mensual              20000 non-null  float64
 1   deuda_total                  20000 non-null  float64
 2   gasto_mensual                20000 non-null  float64
 3   score_crediticio             20000 non-null  float64
 4   edad                         20000 non-null  float64
 5   antiguedad_meses             20000 non-null  float64
 6   frecuencia_compra            20000 non-null  float64
 7   ultima_compra_dias           20000 non-null  float64
 8   num_productos                20000 non-null  float64
 9   hora_registro                20000 non-null  float64
 10  genero_Femenino              20000 non-null  float64
 11  genero_Masculino             20000 non-null  float64
 12  genero_Otro                  20000 non-null  float64
 13  region_Centro                20000 non-null  float64
 14  region_Norte                 20000 non-null  float64
 15  region_Sur                   20000 non-null  float64
 16  estado_civil_Casado          20000 non-null  float64
 17  estado_civil_Divorciado      20000 non-null  float64
```

```
 12  genero_Otro                  20000 non-null  float64
 13  region_Centro                20000 non-null  float64
 14  region_Norte                 20000 non-null  float64
 15  region_Sur                   20000 non-null  float64
 16  estado_civil_Casado          20000 non-null  float64
 17  estado_civil_Divorciado      20000 non-null  float64
 18  estado_civil_Soltero         20000 non-null  float64
 19  canal_registro_App           20000 non-null  float64
 20  canal_registro_Tienda        20000 non-null  float64
 21  canal_registro_Web           20000 non-null  float64
 22  dia_semana_registro_Domingo  20000 non-null  float64
 23  dia_semana_registro_Jueves   20000 non-null  float64
 24  dia_semana_registro_Lunes    20000 non-null  float64
 25  dia_semana_registro_Martes   20000 non-null  float64
 26  dia_semana_registro_Miercoles 20000 non-null float64
 27  dia_semana_registro_Sabado   20000 non-null  float64
 28  dia_semana_registro_Viernes  20000 non-null  float64
 29  tipo_plan                    20000 non-null  float64
 30  uso_app                      20000 non-null  float64
 31  tiene_tarjeta_credito        20000 non-null  float64
 32  abandono                     20000 non-null  float64
dtypes: float64(33)
memory usage: 5.0 MB
```

Figuras 16 y 17: data_transformada.info(). Muestra características del dataset procesado.

La ejecución de data_transformada.info() confirma que todas las características poseen ahora el formato numérico requerido. Adicionalmente, los métodos isnull().sum() y duplicated().sum() retornan 0 en su totalidad, evidenciando el éxito de la imputación de nulos y la eliminación de los registros repetidos.
El uso de data_transformada.describe() verifica el correcto funcionamiento del StandardScaler, mostrando que todas las variables poseen ahora una escala estandarizada con media cercana a 0 y desviación estándar de 1, evidenciando su nueva calidad estadística.

	ingreso_mensual	deuda_total	gasto_mensual	score_crediticio	edad	antiguedad_meses	frecuencia_compra	ultima_compra_dias	num_productos	hora_registro	...
count	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	2.000000e+04	...
mean	-1.644906e-16	3.907985e-17	-1.213252e-16	2.835066e-16	-1.514344e-16	2.531308e-17	6.785683e-17	-1.350031e-16	-1.689315e-16	-3.410605e-17	...
std	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	1.000025e+00	...
min	-3.748982e+00	-4.685669e+00	-2.156942e+00	-2.174782e+00	-1.708799e+00	-1.731512e+00	-1.640247e+00	-1.727105e+00	-1.415323e+00	-1.656725e+00	...
25%	-6.571264e-01	-6.850224e-01	-6.817746e-01	-6.704076e-01	-8.688149e-01	-8.541027e-01	-9.077280e-01	-8.605990e-01	-7.062790e-01	-7.862231e-01	...
50%	2.003240e-02	1.902241e-02	-5.181025e-03	6.737406e-03	-2.883105e-02	2.330692e-02	7.920358e-03	-1.313756e-02	2.765273e-03	-6.080458e-02	...
75%	6.969191e-01	7.062643e-01	6.688074e-01	6.742291e-01	8.671518e-01	8.714696e-01	9.235687e-01	8.724121e-01	7.118095e-01	8.096977e-01	...
max	1.794295e+00	1.782807e+00	2.180274e+00	2.117907e+00	1.707136e+00	1.719632e+00	1.656087e+00	1.729396e+00	1.420854e+00	1.680200e+00	...

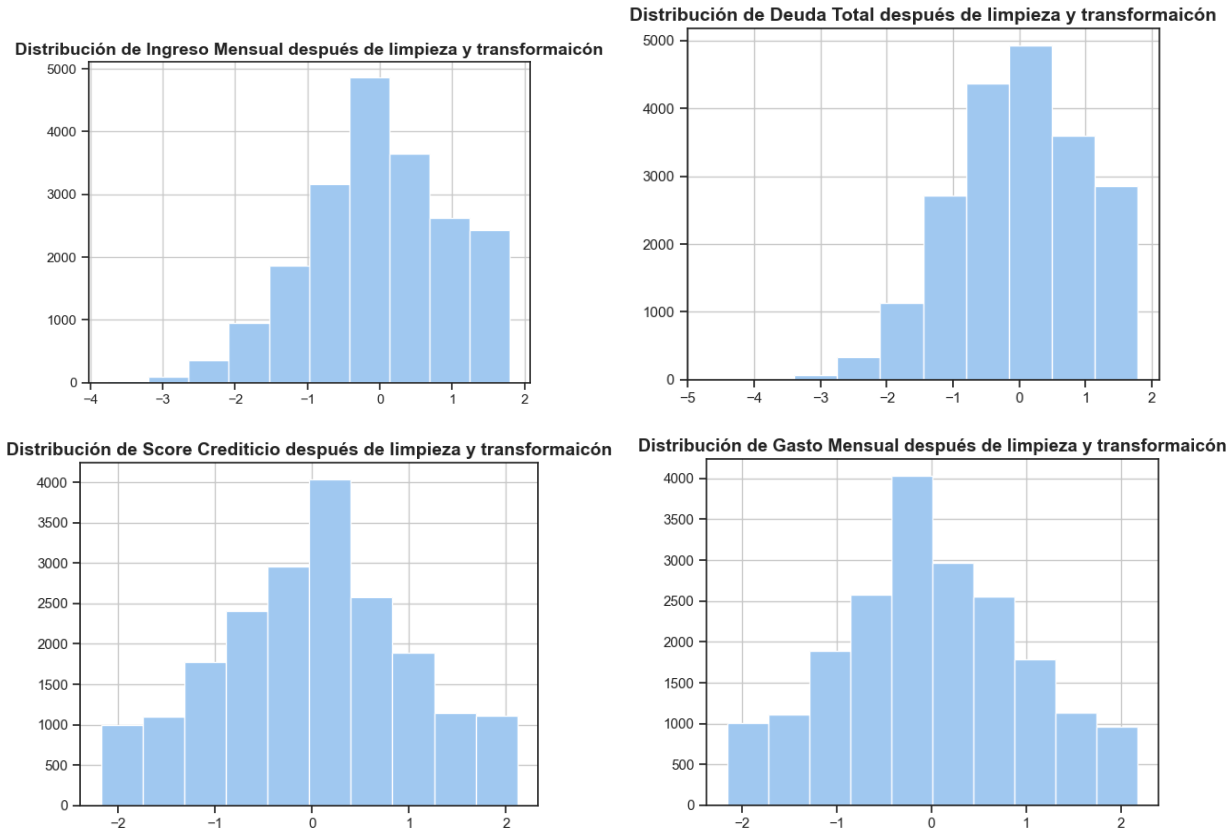
8 rows × 33 columns

dia_semana_registro_Martes	dia_semana_registro_Miercoles	dia_semana_registro_Sabado	dia_semana_registro_Viernes	tipo_plan	uso_app	tiene_tarjeta_credito	abandono
20000.000000	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000
0.143350	0.146700	0.140150	0.141350	1.011050	0.995350	0.500800	0.396750
0.350438	0.353815	0.347151	0.348391	0.814408	0.818695	0.500012	0.489236
0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0.000000	0.000000	0.000000	0.000000	1.000000	1.000000	1.000000	0.000000
0.000000	0.000000	0.000000	0.000000	2.000000	2.000000	1.000000	1.000000
1.000000	1.000000	1.000000	1.000000	2.000000	2.000000	1.000000	1.000000

Figuras 18 y 19: data_transformada.describe() partes 1 y 2.

Control de distribuciones y atípicos

El análisis visual y estadístico de las nuevas distribuciones confirma que la Winsorización trató con éxito los valores extremos en variables como ingreso mensual o gasto mensual, manteniendo la forma de la campana de Gauss o mitigando el sesgo sin pérdida de registros.



Validación de Codificación

Se verificó que la codificación ordinal respetó estrictamente el orden definido para variables Tipo de Plan y Uso de App, transformándolas en secuencias numéricas.

5. Conclusiones y recomendaciones

Reflexión final

El desarrollo de esta fase de preparación de datos nos deja una lección importante como futuros científicos de datos: la calidad del modelo predictivo de machine learning está directamente relacionado con la calidad de los datos usados durante su entrenamiento. Consideramos que se logró cumplir con éxito los objetivos propuestos al inicio: se logró transformar un conjunto de datos crudo que presentaba registros duplicados, valores faltantes y escalas matemáticas dispares, en un dataset robusto, y se lograron alinear las metodologías técnicas con los objetivos del negocio.

Dificultades encontradas

El dataset contiene una mezcla de variables numéricas, categóricas (nominales y ordinales) y booleanas. Esto exigió diseñar un conjunto de pipelines personalizados para garantizar que cada característica recibiera la transformación matemática y codificación necesaria sin generar sesgos.